

CHAPTER 7 — POINTER ARITHMETIC

SLIDE 1 — WHAT IS POINTER ARITHMETIC?

MEET POINTER ARITHMETIC!

Just like normal variables can be used with arithmetic operators, **pointers can also be moved using arithmetic.**

With pointers, we can:

- Move to the **next memory location** 
- Move to the **previous memory location** 
- Move forward by multiple positions
- Move backward by multiple positions

Simple Definition

Pointer arithmetic means performing arithmetic operations on pointers to move between memory locations.

LOOK AT THIS!

Suppose we have an array:

```
None  
int a[] = {10, 20, 30, 40};
```

A pointer can move through the array:

```
None  
10    20    30    40
```

↑
p

After:

None
p++;

The pointer moves to the next element:

None
10 20 30 40
 ↑
 p

EASY ANALOGY

Think of an array like a **row of houses**    .

The pointer is a person standing at one house.

None
 →  →  → 


p++ means:

Move to the next house →

★ REMEMBER!

p++ → NEXT →

p-- → PREVIOUS ←

SLIDE 2 — MOVING A POINTER

🤔 HOW DOES `p++` WORK?

Consider:

None

```
int a[] = {10, 20, 30};  
int *p = a;
```

Initially:

None

```
10    20    30  
↑  
p
```

Now:

None

```
p++;
```

The pointer moves to the next element:

None

```
10    20    30  
      ↑  
      p
```

So:

None

```
*p
```

now gives:

None

20

LOOK AT THIS!

None

```
printf("%d", *p);
```

Output:

None

20

EASY ANALOGY

Imagine walking through numbered rooms 🚪 :

None

🚪 10 → 🚪 20 → 🚪 30



p++ means:

None

🚶 → next room

★ REMEMBER!

p++ moves the pointer to the next element.



SLIDE 3 — MOVING BACKWARD



CAN WE MOVE A POINTER BACK?

Yes! 🎯

We can use:

None

```
p--;
```

1234 EXAMPLE

None

```
int a[] = {10, 20, 30};
```

```
int *p = &a[2];
```

Initially:

None

```
10    20    30
      ↑
      p
```

Now:

None

```
p--;
```

The pointer moves backward:

None

```
10    20    30
      ↑
      p
```

Now:

None

```
*p
```

gives:

None

20

BIG MEMORY TRICK

None

p++ → MOVE FORWARD 

p-- → MOVE BACKWARD 

★ TAKEAWAY

Pointer arithmetic can be used to move through elements.

SLIDE 4 — + POINTER + NUMBER

CAN WE MOVE MORE THAN ONE POSITION?

Yes! 

We can add or subtract a number from a pointer.

For example:

None

```
int a[] = {10, 20, 30, 40};
```

```
int *p = a;
```

Initially:

None

10

20

30

40

↑

p

Now:

None

```
p = p + 2;
```

The pointer moves **2 elements forward**:

None

```
10    20    30    40
      ↑
      p
```

So:

None

```
printf("%d", *p);
```

Output:

None

```
30
```

LOOK AT THIS!

None

```
p      → first element
p + 1   → next element
p + 2   → two elements forward
```

Similarly:

None

```
p - 1 → one element backward
p - 2 → two elements backward
```

EASY ANALOGY

Think of seats in a row :

None



$p + 2$ means:

Move **2 seats forward**  

BIG MEMORY TRICK

$p + n \rightarrow$ MOVE n POSITIONS 

$p - n \rightarrow$ MOVE n POSITIONS 

★ TAKEAWAY

Pointer arithmetic moves according to **elements**, not simply raw bytes.

SLIDE 5 — POINTER ARITHMETIC RECAP

WHAT DID WE LEARN?

1. MOVE FORWARD

None

$p++;$

Moves to the **next element**.

2. MOVE BACKWARD

None

```
p--;
```

Moves to the **previous** element.

3. MOVE MULTIPLE POSITIONS

None

```
p = p + 2;
```

Moves **2** elements forward.

4. MOVE BACKWARD BY MULTIPLE POSITIONS

None

```
p = p - 2;
```

Moves **2** elements backward.

5. ACCESS THE VALUE

None

```
*p
```

Gets the value at the pointer's current position.

ONE BIG MEMORY TRICK!

None

```
p++
```

↓

```
NEXT 
```

```
p--
```

↓
PREVIOUS ←

p + n
↓
n POSITIONS FORWARD →

p - n
↓
n POSITIONS BACKWARD ←

*p
↓
GET VALUE 🔍

★ REMEMBER!

Pointer arithmetic moves the pointer between elements.



QUESTIONS

1 What does `p++` do?

- A) Deletes the pointer
- B) Moves the pointer to the next element
- C) Moves the pointer backward
- D) Changes the value to 0

💡 **Hint:** `++` means increase by one position.

📖 Glossary:

Pointer arithmetic → Using arithmetic operations to move a pointer.

2 What is the output?

None

```
int a[] = {10, 20, 30};
```

```
int *p = a;

p++;

printf("%d", *p);
```

- A) 10
- B) 20
- C) 30
- D) Address of `a`

 **Hint:** `p++` moves to the next element.

 Glossary:

Element → One individual value in an array.

3 What does `*p` give?

- A) The next address
- B) The previous address
- C) The value at the current pointer position
- D) The size of the pointer

 **Hint:** `*` is the dereference operator.

 Glossary:

Dereference → Access the value stored at the pointer's current address.

4 Find the Value After `p + 2`

```
None

int a[] = {100, 200, 300, 400};

int *p = a;

printf("%d", *(p + 2));
```

- A) 100
- B) 200

- C) 300
- D) 400

 **Hint:** $p + 2$ moves two elements forward.

 **Glossary:**

Pointer expression → An expression that performs an operation on a pointer.

5 Trace the Pointer

None

```
int a[] = {10, 20, 30, 40};
```

```
int *p = a;
```

```
p++;
```

```
p++;
```

What is $*p$?

- A) 10
- B) 20
- C) 30
- D) 40

 **Hint:** Each $p++$ moves one position.

 **Glossary:**

Trace → Follow the changes step by step.